```
# 1 import our python library
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
#from xgboost import XGBClassifier

# 2 Import  .csv file for analysis
df = pd.read_csv(r"C:\Users\eunice\PyCharmMiscProject\credit_risk_dataset.csv")

# 3 initial data cleanup
if df.iloc [0].isnull().sum == 0:
   df.columns = df.iloc[0]
   df = df[1:]

if 'ID' in df.columns:
    df.drop(['ID'], axis=1, inplace=True)
else:
    print("Column ID  not found in dataset")

# 4 Handling of missing vaules in .csv file
df_clean = df.dropna(subset=['loan_int_rate']).copy()
df_clean['person_emp_length'] =
df_clean['person_emp_length'].fillna(df_clean['person_emp_length'].median())

# 5 Encode or variables
categorical_columns = ['person_home_ownership', 'loan_intent', 'loan_grade',
'cb_person_default_on_file']
for column in categorical_columns:
    le = LabelEncoder()
    df_clean[column] = le.fit_transform(df_clean[column])

# 6 Final Clean up data
df_clean = df_clean.apply(pd.to_numeric, errors='coerce')
df_clean.dropna(inplace=True)

# 7 split into X and Y variable
X = df_clean.drop(['loan_status'], axis = 1)
y = df_clean['loan_status']

# 8 Baseline Random  forest Model
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.25,
random_state=42)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)

# 9 Print our Data
print("Baseline model performance")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

# 10 Tune model with GridSearchCV
param_grid = {
    'n_estimators': [10, 50, 100],
```

```
    'max_depth': [2, 4, 6, 8, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]

}

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1' ,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print("tuned model performance")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print("Best parameters: ", grid_search.best_params_)

importances = best_model.feature_importances_
feature_names = X.columns
indices = np.argsort(importances)[::-1]

# 11 Use data to plot graph
plt.figure(figsize=(10, 6))
plt.title("Top 10 Feature Importance")
plt.bar(range(10), importances[indices[:10]], align="center")
plt.xticks(range(10), [feature_names[i] for i in indices[:10]], rotation=90)
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.tight_layout()
plt.show()
```